AI-Powered Unified Sri Lankan Tourism Platform Using GraphRAG and LLM Orchestration

INDIVIDUAL THESIS

Deniyawatte D.G.A.D.N.

(IT22895196)

Individual component: Intelligent Tour Package Generation

Group ID: R26-IT-050

Report submitted in partial fulfilment of the requirements for the

BSc (Hons) in Information Technology

Specializing in Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

11 October 2026

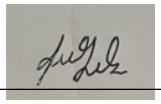
DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Deniyawatte D.G.A.D.N.

Student ID: IT22895196

Signature:  Date: 10/10/2026

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Ms. Dilani Lunugalage		
Co-supervisor: Ms. Dulani Maheshika Jayasinghe		

ABSTRACT

This thesis presents Intelligent Tour Package Generation for a unified Sri Lankan tourism platform. The component interprets natural-language travel requests, validates preferences against graph values, retrieves hotel and activity records, and constructs hotel-centred suggestions. A local language model assists extraction, while graph queries and deterministic rules control the active composition path. A separate hotel-pricing service requests an offer after the traveller selects a hotel and provides an explicit unavailable state when a usable price cannot be obtained. The available hotel catalogue contains 1,830 records across 25 district values and eight accommodation categories. Four existing pricing tests passed, covering stay validation, missing credentials, and a mocked whole-stay offer. These results are focused verification rather than an independent measure of recommendation quality. Source inspection identifies limitations in dietary filtering, district-based activity association, candidate ranking, and repeated day-plan activities. The activity ingestion file is absent from its configured local path, so a complete graph rebuild was not demonstrated. The contribution is an inspectable workflow connecting request interpretation, tourism records, package presentation, and selected-hotel pricing. Future work should establish data provenance, strengthen constraint handling, and evaluate package relevance and feasibility using labelled requests and participant studies.

Keywords: tour package generation, knowledge graph, tourism, preference extraction, hotel pricing

ACKNOWLEDGEMENTS

I thank my supervisor, Ms. Dilani Lunugalage, and co-supervisor, Ms. Dulani Maheshika Jayasinghe, for their guidance. I acknowledge the lecturers and staff of the Sri Lanka Institute of Information Technology and my project teammates for their support in developing and integrating the research components. I am grateful to my family for their encouragement. The technical sources used in this report are listed in the references, and assistance used for report preparation is described in Appendix B.

TABLE OF CONTENTS

1 INTRODUCTION 1
1.1 Overall project context 1
1.2 Individual component context 1
1.3 Problem statement 1
1.4 Research gap 2
1.5 Aim and objectives 2
1.6 Research questions or hypotheses 2
1.7 Scope limitations and assumptions 3
1.8 Individual contribution 3
1.9 Thesis structure 4
2 LITERATURE REVIEW AND RELATED WORK 5
2.1 Review method and scope 5
2.2 Domain and theoretical foundations 5
2.3 Related research 5
2.4 Existing systems products and techniques 6
2.5 Gap synthesis and conceptual framework 6
3 RESEARCH METHODOLOGY 7
3.1 Research design 7
3.2 Data participants or study materials 7
3.3 Experimental or study procedure 7
3.4 Measures and evaluation metrics 8
3.5 Data analysis 8
3.6 Ethics privacy security and responsible AI 8
3.7 Reproducibility and research artefacts 9
4 PRODUCT AND SOLUTION DESIGN 10
4.1 Requirements 10
4.2 Component architecture within the complete solution 10
4.3 Design alternatives and decisions 10
4.4 Detailed design 11
4.5 Standards and professional practice 11
5 IMPLEMENTATION INTEGRATION AND TESTING 12
5.1 Development environment and technology stack 12
5.2 Component implementation 12
5.3 Integration with the other components 12
5.4 Verification and software testing 13

5.5 Deployment and operation 13
6 RESULTS AND VALIDATION 14
6.1 Research results 14
6.2 Product and technical results 15
6.3 User or stakeholder validation 15
6.4 Integration results 16
6.5 Summary of findings 16
7 DISCUSSION 17
7.1 Interpretation of findings 17
7.2 Comparison with literature baselines and existing solutions 17
7.3 Research contribution 17
7.4 Product contribution 18
7.5 Limitations and threats to validity 18
7.6 Implications 18
8 CONCLUSION AND FUTURE DIRECTIONS 20
8.1 Conclusion 20
8.2 Achievement of objectives 20
8.3 Individual research and product contributions 20
8.4 Future directions 21
REFERENCES 22
GLOSSARY 23
APPENDICES 24
Appendix A Reproducibility notes 24
Appendix B Academic integrity and AI use disclosure 25

LIST OF FIGURES

No figures are included in this report.

LIST OF TABLES

Table 6.1. Accommodation catalogue inventory. 14
Table 6.2. Executed pricing verification results..... 15

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial intelligence
API	Application programming interface
CSV	Comma-separated values
JSON	JavaScript Object Notation
LLM	Large language model
RAG	Retrieval-augmented generation
LKR	Sri Lankan rupee
USD	United States dollar

1 INTRODUCTION

1.1 Overall project context

The research project develops a unified platform for tourism in Sri Lanka. Its four components address tour packages, context-aware itinerary optimisation, mobility and vehicle selection, and landmark information. Together, these functions support decisions that travellers otherwise make across separate interfaces. The present thesis concerns Intelligent Tour Package Generation, assigned to Deniyawatte D.G.A.D.N., IT22895196, within group R26-IT-050.

The package component connects accommodation records and activity records through a graph. It translates a natural-language request into structured preferences, retrieves matching records, and presents a small set of hotel-centred suggestions. The implementation and integration described in this report are documented in the R26-IT-050 project repository [1].

1.2 Individual component context

A traveller may describe a destination, food preference, accommodation grade, preferred activities, and trip duration in one sentence. The component provides a way to turn that request into a structured search. Its principal output is a hotel recommendation with related activities and a suggested day plan. Room availability pricing is requested separately after the traveller selects a hotel.

The component uses a local language model for preference extraction, Neo4j for graph retrieval, and deterministic rules for package composition. This division keeps the accommodation and activity entities tied to stored records. It also allows common preferences to be extracted through a fallback when the language-model request fails [2], [3].

1.3 Problem statement

The problem addressed is how to combine conversational input with explicit tourism records so that a traveller receives a coherent accommodation-and-activity suggestion. A free-text request can contain ambiguous language, missing constraints,

or preferences that do not exist in the database. Generating attractive prose alone does not resolve whether a suggested hotel or activity has supporting data.

A second problem is the distinction between a recommendation and a purchasable offer. A hotel can be relevant to the request while no room price is available for the specified dates. The system therefore needs to preserve this distinction and avoid presenting a guessed amount as an available room offer.

1.4 Research gap

The engineering gap investigated here is the connection between flexible request interpretation and constrained package assembly in a Sri Lankan tourism dataset. The work asks whether a graph-backed workflow can expose the records behind a recommendation while supporting natural-language input. This is a project-level research gap; the evidence does not establish that no previous tourism system has implemented similar ideas.

The implementation also exposes an evaluation gap. The repository contains a working design and focused pricing tests, but it does not establish package relevance through an independently labelled request set or a completed traveller study. These missing evaluations shape the claims made in this thesis.

1.5 Aim and objectives

The aim is to develop and examine an intelligent package-generation component that combines hotel and activity information with traveller preferences.

The objectives are to represent tourism entities and their relationships in a graph; extract and validate structured preferences from natural language; retrieve and rank hotel-centred candidates; assemble a readable day plan; request a clearly identified room offer for the selected stay; and connect the selected hotel to the shared mobility workflow. Evaluation distinguishes source-level verification, dataset inspection, automated tests, and future user validation.

1.6 Research questions or hypotheses

RQ1: How can natural-language travel preferences be mapped to values supported by a tourism graph?

RQ2: How can hotel and activity records be combined into traceable package suggestions?

RQ3: How can selected-hotel pricing preserve the difference between unavailable data, sandbox offers, and production responses?

RQ4: What limitations arise when district relationships and simple ranking rules are used to compose a day plan?

The study is exploratory and implementation-oriented. It does not report a statistically tested claim that this method outperforms commercial travel services.

1.7 Scope limitations and assumptions

The scope covers preference extraction, graph ingestion, candidate retrieval, hotel recommendation, basic day-plan composition, selected-hotel pricing, and a handoff to the mobility component. It does not include payment, confirmed booking, a global optimisation of trip cost, or independent route-risk prediction.

The hotel CSV is available locally, but the activity CSV referenced by the ingestion controller is absent at its configured path in the inspected working copy. An existing deployed graph may contain activity data; its contents were not audited for this report. Consequently, end-to-end package generation against a freshly rebuilt graph is not claimed as a completed experiment.

1.8 Individual contribution

The individual contribution is the hotel-centred package-generation workflow: tourism graph preparation, preference interpretation, graph filtering, candidate presentation, and selected-hotel pricing. The frontend also carries the chosen hotel into the group mobility flow. The separate route-risk model and vehicle recommendation logic belong to the mobility component, while the context-aware itinerary optimiser is another group member's component.

This thesis evaluates the code associated with the assigned component. Repository artefacts support the technical descriptions; a source review alone does not establish exclusive authorship of every shared file.

1.9 Thesis structure

Chapter 2 reviews relevant technical foundations. Chapter 3 explains the research and evaluation method. Chapters 4 and 5 describe the design and implementation. Chapter 6 presents the available results, Chapter 7 discusses their meaning and limitations, and Chapter 8 concludes the study. References and reproducibility notes support further review.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review method and scope

The review focuses on retrieval-grounded language systems, structured graph querying, and the project implementation. Primary sources are used for the technical foundations. Repository files are treated as evidence of this implementation, rather than independent evidence that its recommendations improve traveller outcomes.

The review is a focused technical review, not a systematic literature survey. No claim is made about exhaustive database coverage or a formal screening process.

2.2 Domain and theoretical foundations

Retrieval-augmented generation combines model processing with information obtained from an external knowledge source. Lewis et al. investigate this approach for knowledge-intensive language tasks [4]. For this project, the relevant principle is that stored tourism records should constrain the entities returned to the traveller.

A property graph represents entities as nodes and their connections as relationships. In the package component, hotels and activities connect to districts. These relationships support structured retrieval. The current implementation should be understood as graph-grounded retrieval and rule-based composition with language-model-assisted extraction; it does not reproduce a trained neural retriever or an end-to-end RAG training experiment.

2.3 Related research

The RAG literature provides a foundation for separating model knowledge from retrieved evidence [4]. This separation is useful when accommodation details can change independently of a model. However, this project does not experimentally measure a reduction in hallucination. Its stronger, directly inspectable property is that returned hotel and activity objects originate in graph query results.

Neo4j documents parameterised Cypher queries, where values are supplied independently of the query string [5]. The package service follows this approach for filters such as district and activity category. Parameterisation supports a clear

boundary between user preferences and query structure, although it does not by itself establish semantic correctness of those preferences.

2.4 Existing systems products and techniques

Three approaches help explain the design space. A structured form can constrain input but requires the traveller to choose from predefined fields. An unconstrained conversational response accepts flexible language but needs an independent check of the suggested entities. The implemented hybrid interprets language first and then queries stored entities.

A fourth concern is availability pricing. Static catalogue data can identify a hotel without establishing that rooms are available for the requested stay. The project uses a separate provider integration to obtain an offer after selection. This comparison is architectural; no controlled evaluation against named commercial products was performed.

2.5 Gap synthesis and conceptual framework

The conceptual framework contains four stages: interpret the request, constrain the interpretation, retrieve connected entities, and compose a suggestion. Pricing adds a subsequent verification step for the selected hotel. Each stage has a separate failure mode and should be evaluated independently.

The resulting research direction is to measure both preference correctness and package usefulness. A package can contain valid graph entities and still be unsuitable because the district is too broad, the activities are repetitive, or important constraints were discarded. Grounded entities are therefore necessary evidence, but not a complete measure of recommendation quality.

3 RESEARCH METHODOLOGY

3.1 Research design

The study uses an artefact-centred software research approach. The artefact is the package-generation component and its integration with shared tourism services. The evaluation presented here combines source inspection, a reproducible hotel-data inventory, and execution of the existing focused pricing tests.

These methods answer different questions. Source inspection establishes how decisions are implemented. Dataset inspection describes the available input. Unit tests verify specific behaviours under controlled conditions. None of these methods alone measures traveller satisfaction.

3.2 Data participants or study materials

The available hotel dataset contains 1,830 records across 25 distinct district values and eight accommodation category values. The file is named SLTDA_Master_Dataset_Updated_Coords.csv and includes identifiers, room counts, administrative areas, coordinates, grade fields, and food type [6]. The filename is not sufficient to establish the provenance or verification of every enriched field.

The activity importer expects activities_2000_graph_rag.csv. That file was not present at the configured path during inspection, so neither a count of 2,000 activities nor its category distribution is reported as a verified result. There were no human participants in the evaluation documented here.

3.3 Experimental or study procedure

The procedure first traced the request from the frontend through the controller, extractor, validator, graph query, and response construction. It then reviewed how a selected hotel is priced and how the result is displayed. The hotel CSV was counted using a dictionary-based CSV reader, preserving its original records.

The existing liteapi-hotel-pricing.test.js suite was executed with Node's test runner on 11 October 2026. Four tests completed successfully. The provider-response case

uses a mock response, and the missing-credential case returns without a network request. No live provider availability experiment was performed [7].

3.4 Measures and evaluation metrics

The completed measures are dataset row counts, category counts, and pass/fail outcomes for the focused test suite. Future evaluation should measure extraction exact-match accuracy per preference field, unsupported-value frequency, retrieval precision, and constraint satisfaction across a labelled prompt set.

For user-facing quality, useful measures include package relevance ratings, activity repetition, feasible daily duration, and the proportion of requests that receive an informative no-match response. Pricing evaluation should separately measure property-match correctness and provider availability. These are proposed metrics, not reported performance scores.

3.5 Data analysis

Results are analysed descriptively. Accommodation counts show catalogue composition, while source traces explain why a package is selected. The automated tests are interpreted at the level of the assertions they contain. Passing all four tests is evidence for those four cases, not a percentage accuracy for the full package system.

A future comparison should hold the prompt set and graph snapshot constant while comparing structured input, deterministic extraction, and language-model-assisted extraction. Independent relevance labels and repeated runs would help distinguish model variability from retrieval behaviour.

3.6 Ethics privacy security and responsible AI

Travel prompts may contain dates, party sizes, and personal preferences. Operational deployments should minimise unnecessary logging, protect provider credentials, and define retention for request data. The local Ollama endpoint reduces dependence on an external extraction service when configured locally, but the hotel-pricing request still transmits stay information to a provider.

Responsible presentation requires that unavailable prices remain unavailable and that estimated totals identify their exclusions. Dataset provenance and permission to

redistribute enriched hotel and activity data should be documented before a public release. No participant consent process is claimed because no participant study is reported.

3.7 Reproducibility and research artefacts

The reproducibility artefacts are the repository controller and services, frontend screens, hotel CSV, and pricing tests. A repeatable full-system evaluation additionally requires the activity dataset, a dated Neo4j graph snapshot, model version, dependency lockfiles, and environment configuration without secrets.

The model name defaults to llama3 and the local endpoint defaults to port 11434. Recording the exact downloaded model revision is important because a name alone does not identify immutable weights. Appendix A lists the operational evidence needed for a future reproduction.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

The functional requirements are to accept a travel request, validate stay details, extract supported preferences, return matching hotel-and-activity candidates, produce a day plan, request selected-hotel pricing, and show clear empty or unavailable states. The optional vehicle flow must preserve the selected hotel and pricing context.

Quality requirements include traceability to graph records, bounded recommendation lists, clear currency labelling, and graceful handling of provider failures. Feasible daily scheduling and verified geographical proximity are desirable requirements that are not fully satisfied by the present composition rules.

4.2 Component architecture within the complete solution

The frontend sends the prompt and stay details to an Express controller. The extraction service reads allowable graph values, requests structured JSON from Ollama, and passes the result through validation. The package service executes a parameterised graph query. The controller ranks the returned candidates and builds a day-wise response [1], [2], [3].

Hotel selection triggers a separate price request. The pricing service validates the stay, retrieves the selected catalogue hotel, searches the provider response for a matching property, and maps the offer. This separation allows recommendations to remain visible when pricing is unavailable. The frontend can then pass the chosen package into the mobility workflow [8].

4.3 Design alternatives and decisions

The graph model makes the relationship between hotels, districts, and activities explicit. Schema-aware extraction limits several output fields to values already represented in the graph. A deterministic fallback supports familiar destination and category terms if the model response is unavailable or invalid.

The controller uses simple ranking and composition rules so that the response structure is predictable. This improves inspectability but limits personalisation. The current tie-breaker uses room count, which should not be interpreted as an independently validated measure of hotel quality or available rooms.

4.4 Detailed design

Hotels carry identifiers, names, coordinates, category, grade, food type, and room count. District, administrative division, and administrative area nodes organise location information. Activities carry identifiers, categories, descriptions, opening hours, price level, audience suitability, duration, and coordinates [3].

The activity importer creates NEAR_ACTIVITY relationships between hotels and activities in the same district. The relationship name expresses an intended association; the implementation does not calculate distance or journey time. A large district can therefore connect a hotel to an activity that is inconvenient to reach.

The ranking sorts retrieved candidates by matching activity count and then room count. The graph query limits candidates to ten and caps each activity collection at five before this sort. Thus the displayed top three are the highest-ranked items within a limited retrieved subset, not a proven global optimum.

4.5 Standards and professional practice

The API uses JSON request and response objects and explicit validation outcomes. Parameterised Cypher separates preference values from query structure. The code closes graph sessions in finally blocks, supporting resource cleanup during failures.

Professional practice also requires accurate labels. A price must identify its currency and environment, and a suggested plan must not imply a confirmed reservation. IEEE-style numbered references in this thesis link implementation statements to the relevant repository artefacts and primary documentation.

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development environment and technology stack

The implementation uses JavaScript on a Node.js and Express backend, Neo4j for graph storage, and Ollama for local model inference. The client screens use React Native components. CSV ingestion supplies hotel and activity entities; a separate HTTP integration requests hotel rates.

This stack reflects the shared project architecture. The package component does not train a new language model. Its research contribution concerns orchestration, validation, retrieval, and presentation around existing inference and data services.

5.2 Component implementation

The extraction prompt lists allowed graph values and asks for eight fields: district, hotel category, grade, food type, duration, activity category, price level, and audience suitability. The validator uses supported terms from the original prompt as fallbacks, normalises common food and grade expressions, and clears several values that do not match the graph schema [2].

Clearing an unsupported value can broaden retrieval. For example, a destination that fails validation can become a null filter rather than an explicit request for clarification. The fallback also relies on lexical matching, so its success with unfamiliar paraphrases or multilingual requests is not established.

The graph query retrieves hotels connected to matching activities. The controller keeps up to three candidates and assigns activities to days. If duration exceeds the number of retained activities, the modulo-based assignment repeats activities. This is a basic schedule representation and does not optimise travel time, opening hours, or daily workload.

5.3 Integration with the other components

The input screen collects a prompt, check-in and check-out dates, adults, and room quantity. The results screen displays hotel candidates, activities, and suggested days,

then requests a price for the selected hotel. It renders an unavailable state when the provider cannot supply a usable offer [8].

When the traveller enables vehicle planning, the selected hotel and package context are sent to the shared Main flow. A combined estimate is shown only when the hotel offer is in LKR and a vehicle estimate is available. The displayed combined total explicitly excludes activity charges. This is an integration of estimates rather than an all-inclusive package price.

5.4 Verification and software testing

The focused test suite checks stay-length calculation, rejection of selected invalid stay inputs, missing-credential behaviour, and mapping of a mocked provider total. All four tests passed in the execution recorded for this thesis [7].

Important untested cases remain: semantic preference errors, graph unavailability, incorrect provider property matches, no matching activities, unstable ordering, and long requested durations. In particular, the vegetarian query uses a substring test for "veg", which can also match "non-veg". This is a source-identified filtering limitation and should be corrected and covered by a regression test.

5.5 Deployment and operation

A deployment requires the Express backend, an accessible Neo4j instance, the expected graph content, and the configured Ollama model. Hotel-price requests additionally require provider credentials and connectivity. The activity source file must be restored or an independently verified graph snapshot supplied before a fresh ingestion can be reproduced.

Operational readiness should include bounded prompt and duration inputs, client-side request timeouts for provider calls, protected administrative ingestion routes, and monitoring of unavailable states. These are deployment recommendations; this thesis does not certify a production deployment or load-test result.

6 RESULTS AND VALIDATION

6.1 Research results

The available results demonstrate an implemented separation between language interpretation, graph retrieval, deterministic composition, and selected-hotel pricing. This supports a traceable technical workflow. It does not yet demonstrate superiority over a baseline recommender.

The hotel inventory contains 1,830 rows. Guest houses are the largest category at 893 rows, followed by home stays at 378 and bungalows at 348. These three categories account for 1,619 rows. Counts describe catalogue representation, not verified availability or tourism demand.

Table 6.1. Accommodation catalogue inventory.

Category	Records
Guest Houses(1)	893
Home Stay Units	378
Bungalows	348
Classified Hotels(1-5 Star)	141
Boutique Villas	36
Boutique Hotels	28
Heritage Bungalow	4
Heritage Homes	2
Total	1830

6.2 Product and technical results

The implemented product produces structured hotel recommendations with related activities and a day plan. It exposes a separate selected-hotel pricing state and supports a handoff to mobility planning. These are source-verified capabilities; a completed live UI experiment is not inferred from the code alone.

Table 6.2. Executed pricing verification results.

Existing test case	Observed result
Complete stay length	Passed: three-night calculation
Selected invalid stay inputs	Passed: rejected
Missing provider credentials	Passed: unavailable, no invented total
Mocked whole-stay offer	Passed: mapped total and per-room-night estimate

The mocked offer contains USD 300 for three nights and two rooms. The service derives USD 50 per room per night by dividing the whole-stay total by nights and room quantity. The request in that test asks for LKR, but the response fixture supplies USD. Preserving the returned currency prevents the amount from being silently treated as a converted LKR value.

The four passing tests establish these specific behaviours. They do not verify live market prices, successful booking, package relevance, or response time under concurrent demand.

6.3 User or stakeholder validation

No completed traveller or stakeholder study is available for this component. Accordingly, satisfaction scores, time savings, and acceptance rates are not presented as findings.

A suitable follow-up study would ask participants to complete realistic hotel-and-activity searches, compare the returned alternatives, and explain unmet preferences. Tasks should include ambiguous destinations, dietary requirements, limited activity coverage, and unavailable pricing. Consent and an appropriate data-retention process would be required before collecting participant responses.

6.4 Integration results

The frontend contains an explicit handoff from the selected hotel to the mobility workflow and a summary that combines compatible hotel and vehicle amounts. This is evidence of an implemented interface contract. It is not equivalent to a completed end-to-end test across all four project components.

A robust integration evaluation should retain the original request, selected hotel identifier, provider currency, vehicle result, and exclusion of activity costs. It should also verify that a failed price request does not replace a previously selected package with unrelated data.

6.5 Summary of findings

The strongest verified findings are the hotel-data inventory and the four passing pricing tests. Source inspection supports the component architecture and identifies concrete weaknesses in filtering, ranking, and day-plan feasibility.

The current evidence supports describing the work as an implemented prototype with focused verification. Independent package-quality evaluation, activity-data provenance, and live integration measurements remain open.

7 DISCUSSION

7.1 Interpretation of findings

The results show why separating retrieval and pricing is useful. A valid graph match can be presented without fabricating a room amount, and a provider offer can retain its own currency. The design therefore makes important uncertainty visible to the traveller.

At the same time, a graph-supported entity is not automatically a suitable recommendation. District-level association and capped activity collections can produce a plausible-looking plan without establishing practical proximity or diversity. The method's value lies in its inspectable structure; its usefulness must still be measured against traveller needs.

7.2 Comparison with literature baselines and existing solutions

The component applies the broad retrieval-grounding principle discussed in Chapter 2, but its active generation path is deterministic. The separate itinerary text-generation helper is not invoked by the package controller in the inspected implementation. It would therefore be inaccurate to attribute current day-plan quality to an evaluated generative itinerary model.

The use of fixed parameterised queries provides a clearer retrieval contract than allowing unrestricted model-generated query text. However, correct query syntax does not repair an incorrectly extracted preference. Evaluation should therefore test the extraction and retrieval stages separately.

7.3 Research contribution

The research contribution is an operational formulation of hotel-centred package generation for the project's Sri Lankan tourism records. It connects schema-constrained language interpretation with explicit graph relationships and makes selected-hotel pricing a separate stage.

The analysis contributes a set of testable questions about constraint retention, geographical association, activity diversity, and offer identity. No new model

architecture, global optimisation guarantee, or statistically demonstrated recommendation improvement is claimed.

7.4 Product contribution

The product contribution is a connected interaction from request entry to alternative hotel suggestions, related activities, a day plan, and selected-hotel pricing. The optional mobility handoff extends the package context into the group platform while preserving responsibility boundaries between components.

The component also provides useful failure states: no matching stays and unavailable pricing. These responses help avoid presenting a fabricated package or amount when supporting data is missing.

7.5 Limitations and threats to validity

Several limitations affect validity. The activity ingestion file is missing from its expected path, which prevents a fully reproducible rebuild from the inspected files. The hotel dataset's enriched fields require provenance checks. No labelled prompt benchmark or independent participant evaluation is available.

The query's vegetarian substring condition can admit non-vegetarian values. Nulling an unsupported preference can silently relax a request. Candidate limiting occurs before application ranking, and the activity cap compresses the ranking signal. District-based links do not guarantee short travel distances, and repeated activities do not establish a feasible itinerary.

Provider hotel matching relies on normalised names and substring similarity. It can confuse similarly named properties. The production flag is inferred when the sandbox flag is not true; absence of that flag is not independent confirmation of live-market validity. These issues limit the strength of pricing and recommendation claims.

7.6 Implications

Future development should prioritise correctness before richer generated descriptions. Strict preference validation, transparent relaxation, verified property

identifiers, and geographic distance constraints would address risks that directly affect user decisions.

For research, a fixed benchmark should compare extraction methods and quantify how each constraint changes retrieval. For practice, a recommendation should show the reasons it matches and disclose unresolved conditions. A complete package total should only be introduced when all included costs and currencies are supported.

8 CONCLUSION AND FUTURE DIRECTIONS

8.1 Conclusion

This thesis presents Intelligent Tour Package Generation within the AI-Powered Unified Sri Lankan Tourism Platform Using GraphRAG and LLM Orchestration. The component combines language-assisted preference extraction, structured graph retrieval, simple package composition, and selected-hotel pricing.

The verified evidence comprises an inventory of 1,830 hotel records, source-level analysis, and four successful focused pricing tests. The work establishes a prototype and a clear path for further evaluation. It does not yet establish independent recommendation accuracy, traveller satisfaction, or production booking reliability.

8.2 Achievement of objectives

The objectives of modelling hotel relationships, extracting structured preferences, retrieving candidates, composing suggested days, and implementing selected-hotel pricing are represented in the source. The frontend also implements the selected-hotel handoff to mobility planning.

The objective of comprehensive evaluation is only partly achieved. Focused pricing behaviour is verified, while end-to-end graph reproduction, package relevance, geographical feasibility, and user validation require additional evidence. Stating this boundary allows subsequent work to target the most consequential gaps.

8.3 Individual research and product contributions

The individual component brings hotel and activity selection into a single graph-backed workflow. Its contribution includes preference handling, candidate retrieval and ranking, package response composition, and the pricing interaction. Integration with the shared platform allows accommodation decisions to inform later mobility choices.

The account in this report remains limited to this assigned component. The landmark classifier, route-risk prediction, vehicle ranking, and separate context-aware itinerary optimisation are group contributions outside this individual scope.

8.4 Future directions

The next priorities are to restore and document the activity dataset, correct dietary filtering, validate requested durations, and introduce explicit clarification for unsupported preferences. Hotel-to-activity links should incorporate measured distance or travel time. Candidate ranking should be evaluated before expanding its complexity.

Further work should add provider property identifiers, stricter calendar validation, bounded HTTP requests, and explicit environment verification. A labelled request benchmark and an ethically conducted user study should then assess relevance, constraint satisfaction, and usability. These steps would support a stronger final evaluation without relying on unsupported accuracy claims.

REFERENCES

[1] R26-IT-050 Project Team, “Package generation controller and retrieval service,” backend/src/controllers/hotelData.controller.js and backend/src/services/package.service.js, R26-IT-050 project repository, source code, 2026. Inspected: Oct. 11, 2026.

[2] R26-IT-050 Project Team, “Preference extraction, validation, prompts, and Ollama integration,” backend/src/services/llm/ and backend/src/services/schema.service.js, R26-IT-050 project repository, source code, 2026. Inspected: Oct. 11, 2026.

[3] R26-IT-050 Project Team, “Hotel and activity graph ingestion,” backend/src/services/hotelGraph.service.js and activityGraph.service.js, R26-IT-050 project repository, source code, 2026. Inspected: Oct. 11, 2026.

[4] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in Advances in Neural Information Processing Systems, vol. 33, 2020. [Online]. Available: https://arxiv.org/abs/2005.11401

[5] Neo4j, “Parameters,” Cypher Manual. Accessed: Oct. 11, 2026. [Online]. Available: https://neo4j.com/docs/cypher-manual/5/syntax/parameters/

[6] R26-IT-050 Project Team, “Hotel catalogue,” backend/src/data/SLTDA_Master_Dataset_Updated_Coords.csv, R26-IT-050 project repository, dataset, 2026. Inspected: Oct. 11, 2026.

[7] R26-IT-050 Project Team, “Hotel pricing service and focused tests,” backend/src/services/liteApiHotelPricing.service.js and backend/tests/liteapi-hotel-pricing.test.js, R26-IT-050 project repository, 2026. Tests executed: Oct. 11, 2026.

[8] R26-IT-050 Project Team, “Hotel request and results screens,” frontend/src/screens/HotelHomeScreen.js and HotelResultsScreen.js, R26-IT-050 project repository, source code, 2026. Inspected: Oct. 11, 2026.

GLOSSARY

Candidate: A hotel and related activities returned by retrieval.

Constraint: A condition specified by the traveller, such as destination or food preference.

Grounding: Connecting an output to stored source records.

Property graph: A representation of entities, attributes, and relationships.

Sandbox offer: A development response that should not be treated as a live market quote.

Whole-stay total: The returned accommodation amount for the requested rooms and nights.

APPENDICES

Appendix A Reproducibility notes

The package request path is implemented in backend/src/controllers/hotelData.controller.js. Graph retrieval is defined in backend/src/services/package.service.js. Preference extraction and validation are under backend/src/services/llm/. The hotel and activity graph services define ingestion relationships.

The hotel CSV used for the inventory is backend/src/data/SLTDA_Master_Dataset_Updated_Coords.csv. A reproducible rebuild also needs the activity file referenced by the controller, a documented graph snapshot, the exact Ollama model, and environment configuration with secrets removed.

The focused verification command is node --test backend/tests/liteapi-hotel-pricing.test.js, executed from the repository root. Four tests passed on 11 October 2026. The provider-mapping test uses a mock response; no live booking or provider quote was obtained.

A future benchmark should record each prompt, expected preferences, retrieved hotel identifiers, selected activities, graph version, model version, latency, and independent relevance judgement. No screenshots are included in the appendix.

Appendix B Academic integrity and AI use disclosure

Generative AI assistance through Codex was used to inspect repository artefacts, organise the report using the supplied thesis template, draft technical explanations, format references, count hotel records, execute the existing focused tests, and verify document layout. No participant results, live room prices, or application screenshots were fabricated.

The candidate should verify the individual contribution statement before signing. Additional AI use during research or software development should be disclosed according to institutional requirements. Candidate and supervisor signatures must be completed by the relevant individuals.